

Тема

GitLab CI. Создание пайплайнов, Внутренние переменные, Артефакты



Константин Брюханов

Lead DevSecOps **ВТБ**

- ❑ 7 лет администрирования, из них 3 в DevOps
- ❑ Получаю PhD с диссертацией о проблемах внедрения DevOps
- ❑ Руководитель и лектор курсов по DevOps
- ❑ Ведущий подкастов об IT и образовании
- ❑ Несу DevOps в массы



@deusops

Содержание урока

- ❑ Сохранение результатов работы
- ❑ Знакомство с Services
- ❑ Задание дополнительных условий
- ❑ Знакомство с Container Registry

Внутренние переменные

Предопределенные переменные GitLab CI

Позволяют параметризовать и конфигурировать пайплайны и задачи в них, используя уникальные переменные, определяемые для каждой задачи

- ★ **CI_COMMIT_REF_NAME** – Тэг или ветка, для которой запустился пайплайн
- ★ **CI_COMMIT_SHORT_SHA** – Короткий хэш коммита
- ★ **CI_PROJECT_NAME** – Имя проекта в Gitlab

Предопределенные переменные GitLab CI

```
test1:
  stage: test
  script:
    - echo ${CI_PROJECT_NAME}
  tags:
    - shell

test2:
  stage: test
  script:
    - echo ${CI_COMMIT_SHORT_SHA}
  tags:
    - windows
```

```
24 $ echo ${CI_PROJECT_NAME}
25 git-lesson
27 Cleaning up file based variables
```

```
24 $ echo ${CI_COMMIT_SHORT_SHA}
25 3ed05394
27 Cleaning up file based variables
29 Job succeeded
```

Сохранение результатов

Работа с артефактами

cache

Сохранение результатов работы раннера между задачами одного этапа

artifacts

Сохранение результата сборки и работы раннера для последующего использования. Можно скачать через интерфейс. Используется для передачи состояния между этапами

cache

```
image: node:latest

# Cache modules in between jobs
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .npm/

before_script:
  - npm ci --cache .npm --prefer-offline

test_async:
  script:
    - node ./specs/start.js ./specs/async.spec.js
```

- ★ Можно задавать любой ключ (можно не задавать)
- ★ Можно использовать только в рамках одного раннера
- ★ Нельзя скачать в виде артефакта

artifacts

```
pdf:  
  script: xelatex mycv.tex  
  artifacts:  
    paths:  
      - mycv.pdf  
    expire_in: 1 week
```

Job artifacts

The artifacts will be removed in 6 days

Keep

Download

Browse

Job details

Duration: 5 minutes 4 seconds

Finished: about 5 hours ago

Runner: #44028

Raw

Erase

- ★ Множество вариаций конфигурирования
- ★ Доступно между раннерами
- ★ Можно скачать через UI
- ★ Можно выкладывать как репорты

```
job:  
  artifacts:  
    when: on_failure
```

artifacts

Artifacts / selenium

 Download artifacts archive

Name	Size
 ..	
 selenium	
 some-log.json	17 Bytes
 some-page.html 	4 Bytes
 some-text.txt	4 Bytes

artifacts

PipelinesJobsEnvironmentsCycle Analytics					
All832	Running0	Branches	Tags	Run pipelineCI Lint	
Status	Pipeline	Commit	Stages		
passed	#6484362 by APIlatest	57a5b7d5 Merge branch 'revert-caa47ca...	✓!✓	00:13:35 5 hours ago	Download compile artifacts
passed	#6484314 by APIlatest	57a5b7d5 Merge branch 'revert-caa47ca...	✓!✓	00:14:53 5 hours ago	Download compile artifacts

https://example.com/<namespace>/<project>/-/jobs/artifacts/<ref>/download?job=<job_name>

Services

Зачем нужны сервисы



Сервис — это дополнительный контейнер, который поднимается вместе с основным, в момент выполнения задачи.

Контейнеры связываются между собой (links).
К сервису имеется доступ во время сборки.

Пример сервиса postgresql

```
1  services:
2    - postgres
3
4  variables:
5    POSTGRES_DB: custom_db
6    POSTGRES_USER: custom_user
7    POSTGRES_PASSWORD: custom_pass
8
9  connect:
10   image: postgres
11   script:
12     - export PGPASSWORD=$POSTGRES_PASSWORD
13     - psql -h "postgres" -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT 'OK' AS status;"
```

```
$ psql -h "postgres" -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c "SELECT 'OK' AS status;"
status
-----
OK
(1 row)
Job succeeded
```

Применение различных версий

```
default:
  before_script:
    - bundle install

test:2.6:
  image: ruby:2.6
  services:
    - postgres:11.7
  script:
    - bundle exec rake spec

test:2.7:
  image: ruby:2.7
  services:
    - postgres:12.2
  script:
    - bundle exec rake spec
```

Можно использовать разные версии
одного сервиса для приложения.
Например, для тестирования.

Конфигурирование

```
default:
  image:
    name: ruby:2.6
    entrypoint: ["/bin/bash"]

  services:
    - name: my-postgres:11.7
      alias: db-postgres
      entrypoint: ["/usr/local/bin/db-postgres"]
      command: ["start"]

  before_script:
    - bundle install

  test:
    script:
      - bundle exec rake spec
```

Можно задать параметры для сервисного контейнера, как для любого обычного контейнера. Например, переопределить имя и стартовые команды.

Переменные

```
variables:
  HTTPS_PROXY: "https://10.1.1.1:8090"
  HTTP_PROXY: "https://10.1.1.1:8090"
  POSTGRES_DB: "my_custom_db"
  POSTGRES_USER: "postgres"
  POSTGRES_PASSWORD: "example"
  PGDATA: "/var/lib/postgresql/data"
  POSTGRES_INITDB_ARGS: "--encoding=UTF8 --data-checksums"

services:
  - name: postgres:11.7
    alias: db
    entrypoint: ["docker-entrypoint.sh"]
    command: ["postgres"]

image:
  name: ruby:2.6
  entrypoint: ["/bin/bash"]

before_script:
  - bundle install

test:
  script:
    - bundle exec rake spec
```

Можно передавать переменные как для вашего контейнера, так и для сервисов, что позволяет упростить конфигурирование.

Классический пример: **dind**

```
1 docker_build:
2   stage: build
3   image: docker:stable
4   services:
5     - docker:dind
6   script:
7     - docker login -u gitlab-ci-token -p ${CI_JOB_TOKEN} ${CI_REGISTRY}
8     - docker build -t ${CI_REGISTRY}/${CI_PROJECT_PATH}/${CI_PROJECT_VERSION} .
9     - docker push ${CI_REGISTRY}/${CI_PROJECT_PATH}/${CI_PROJECT_VERSION}
10    - docker tag
11      ${CI_REGISTRY}/${CI_PROJECT_PATH}/${CI_PROJECT_VERSION}
12      ${CI_REGISTRY}/${CI_PROJECT_PATH}/${CI_PROJECT_VERSION}
13    - docker push ${CI_REGISTRY}/${CI_PROJECT_PATH}/${CI_PROJECT_VERSION}
14   tags:
15     - docker
```

Используется для доступа
к демону докера изнутри
контейнера

**Дополнительная
гибкость пайплайна**

Дополнительные параметры задач

before_script

after_script

То же самое, что и script -
но выполняется либо
перед, либо после него

only

when

rules

Правила для параметризации
задач: можно задать условия
при которых пайплайн будет
запускаться

only и when

```
deploy_job:
  stage: deploy
  script:
    - make deploy
  when: manual

cleanup_job:
  stage: cleanup
  script:
    - cleanup after jobs
  when: always
```

when – условие запуска задачи

```
job:
  # use regexp
  only:
    - /^issue-.*$/
  # use special keyword
  except:
    - branches
```

only – ограничение выполнения по условиям

```
job:
  # use special keywords
  only:
    - tags
    - triggers
    - schedules
```

rules

```
docker build:  
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .  
  rules:  
    - if: '$CI_COMMIT_BRANCH == "master"'  
      when: delayed  
      start_in: '3 hours'  
      allow_failure: true
```

**Задание дополнительных
условий для джобы**

```
rules:  
  - if: '$CI_PIPELINE_SOURCE == "push"'  
  - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
```

```
rules:  
  - if: '$CUSTOM_VARIABLE == "false"'  
    when: never  
  - when: always
```

rules

```
docker build:  
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .  
  rules:  
    - changes:  
      - Dockerfile  
      when: manual  
      allow_failure: true
```

**Запуск джобы в случае
изменения файлов**

```
job:  
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .  
  rules:  
    - exists:  
      - Dockerfile
```

**Запуск джобы в случае
наличия файлов**

Package & Container Registry

Что такое Gitlab Registry

Дополнительная функция гитлаба, которая позволяет хранить пакеты приложений или Docker-образы для проектов, с упрощенным доступом к ним

- ★ Для каждого проекта - свой реестр
- ★ Доступ по учетным данным в Gitlab
- ★ Доступ через CI по токену



Пример реестра

Для проекта можно иметь несколько разных образов, например, по одному на компонент

Каждый образ можно (даже нужно) отдельно версионировать разными тэгами

Container Registry

 5 Image repositories  Expiration policy will run in about 23 hours

With the GitLab Container Registry, every project can have its own space to store images

Image Repositories

vtb-web1/ib-core/ib-core-pschannel 

 35 Tags

vtb-web1/ib-core/ib-core-auth 

 31 Tags

vtb-web1/ib-core/ib-core-soap 

 5 Tags

vtb-web1/ib-core/ib-core-soapui 

 2 Tags

vtb-web1/ib-core/ib-core-superauth 

 2 Tags

LIVE



Домашнее задание

Цели вебинара

Проверка достижения целей

#1 Изучили понятие Services и поняли зачем и когда его применять

#2 Изучили возможности сохранения результатов работы пайплайнов

#3 Научились работать с условиями для задач



Домашнее задание

#1 Создать пайплайн для сборки и тестирования приложения

#2 Добавить в пайплайн условия для сборки **если менялся**
файл `pom.txt`

#3 Добавить шаг `publish` и сохранить результат как
артефакт в gitlab-ci

